

Inside Steam Phishing: Credential Harvesting Mechanics and Evasion Infrastructure

Sara Shakeel, Enrico Branca, Natalia Stakhanova

Department of Computer Science, University of Saskatchewan, Canada

Abstract. Phishing campaigns targeting Steam’s virtual asset marketplace exploit a sophisticated and resilient technical infrastructure designed to maximize reach while evading detection. In this paper, we dissect the operational mechanics behind these campaigns, revealing how attackers sustain prolific activity through layered evasion techniques. Over a 125-day longitudinal study monitoring 1,250 Steam groups and 607,832 messages, we identified 127,155 scam links deployed through a coordinated techniques including URL shorteners, typosquatted domains, and advanced Javascript obfuscation enabling scam pages to resist automated takedown and static analysis. We further characterize the campaign’s organizational structure, in which a concentrated actor set (360 accounts, with just 19 responsible for 80% of all scam posts) achieves outsized distribution through high-frequency automation. Together, these technical characteristics constitute a self-reinforcing system that exploits the absence of proactive platform-level enforcement on Steam. Our findings show that Steam phishing not as opportunistic abuse, but a technically mature threat sustained by deliberate infrastructure choices.

1 Introduction

Steam is the dominant gaming distribution platform, occupying approximately 75% of the global gaming market. Its integrated virtual economy, player-to-player item trading, in-game purchases, and a community marketplace that processed over \$500 million in transactions in 2024 alone, make it a uniquely high-value target for credential-harvesting campaigns.

Although security researchers have examined many threats prevalent in video games, including cheating [11], social engineering attacks [8, 1, 10], and phishing [30, 12, 29], the scale of fraud and scams targeting the online gaming industry continues to grow. Steam remains among the most heavily targeted gaming platforms for phishing, ranked among the top five most-impersonated brands globally [6]. Mitigating these threats is difficult without first understanding the mechanics by which attacks are conducted and the evasion techniques that allow them to persist undetected.

In this work, *we address this gap by providing an empirical analysis of the credential harvesting mechanics and evasion mechanisms underlying phishing attacks targeting the Steam ecosystem.* Our study focuses on the artifacts exposed

across the attack, from the initial user click on a malicious message URL to credential harvesting and delivery to the attacker.

While phishing has been an active area of research, with numerous studies examining its nature [19, 20, 13] and proposing detection and mitigation techniques [18, 34, 2], the post-click attack mechanics, what the phishing infrastructure does once a victim arrives and submits credentials, remain largely unexamined.

We focus on this critical phase, examining the mechanics and evasion infrastructure activated to collect a victim’s credentials. Generic phishing research does not readily transfer to this context as Steam’s authentication model, combining session tokens, two-factor authentication, and a proprietary login flow, creates a distinct attack surface that requires platform-specific phishing kits and active server-side coordination, which are not captured by general-purpose phishing detection frameworks. Unlike generic phishing, Steam-targeted phishing requires an active, stateful relay that mimics Steam’s real authentication protocol in real time. Because Steam Guard’s two-factor authentication is embedded directly in the login flow, phishing kits must forward credentials to Steam’s actual servers, interpret live response codes, and dynamically prompt victims for time-sensitive 2FA codes before the authentication window expires. This transforms the attack from passive data harvesting into a functional man-in-the-middle proxy against Steam’s backend, often incorporating Steam-specific elements (e.g., trade offers, QR-code logins, and marketplace lures). Because this malicious behavior occurs server-side and depends on multi-round protocol interaction rather than a static front-end artifact, it evades detection frameworks designed to analyze phishing pages in isolation. This protocol-driven attack surface motivates our focus on the post-click infrastructure and mechanics unique to Steam phishing.

To support our analysis, we collected over 607,832 messages exchanged by users in Steam group chats over 125 days, from which we identified and extracted URLs corresponding to active phishing campaigns. We further intercepted network communication and deobfuscated the Javascript code that served as the primary vehicle for credential exfiltration.

Our contributions are as follows:

- We present a large-scale measurement study of the end-to-end mechanics in Steam phishing attacks. We characterize the scale and organization of the underlying phishing infrastructure, revealing extreme concentration among a small number of accounts and source links, sustained link persistence over multiple weeks, and extensive destination diversity behind a limited set of entry-point URLs indicating a deliberate evasion strategy designed to bypass URL-based blocklists.
- We reconstruct the complete phishing attack chain through passive traffic interception and manual JavaScript deobfuscation, uncovering a token-based phishing kit architecture that assigns victims unique session encryption keys and conceals all backend API communication through runtime XOR encryption.

Disclosure In accordance with responsible disclosure practices, we reported our findings to Valve and, at the time of writing, have not received a response.

In Section 2, we provide an overview of prior security research on phishing attacks in video games and show that Steam, and the broader PC gaming market, remains largely unexplored in this regard. Section 3 details our methodology for collecting Steam group chat messages and analyzing the underlying mechanics of the observed phishing attacks. We present our findings in Section 4 and 5, and conclude with a discussion of the implications of these results in Section 6.

2 Related work

Research on attacks in online gaming ecosystems has historically emphasized cheating mechanics and fraud within player-driven economies [1, 8, 10]. However, empirical investigations into coordinated phishing campaigns, particularly those exploiting modern platform features such as group chats, remain limited.

Recent surveys by Rishiwal et al. [26] and Parizi et al. [24] provide broad overviews of security threats and mitigations across online gaming platforms, while Grandhi and Galimotu [5] and Meyer [17] established foundational taxonomies of social engineering tactics and trading scams in online gaming, along with proposed developer-side mitigations.

A substantial body of work has examined malicious activity in online games. Han et al. [7] conducted a systematic literature review of cheating techniques and their detection methods in online gaming, cataloguing both client- and server-side exploits. Karkallis et al. [11] analyzed game cheating communities to detect injectors, i.e., malicious tools used to manipulate game processes, demonstrating how underground communities organize around the distribution of game-targeting malware. Hoang [9] further examined game hacking techniques and the effectiveness of anti-cheat mechanisms, illustrating the breadth of the attack surface in modern gaming platforms. Oh et al. [21] proposed an automatic detection system for compromised MMORPG accounts, identifying behavioral anomalies indicative of account takeover. Tricomi et al. [33] investigated privacy threats in Dota 2, demonstrating how platform features can expose sensitive player information.

Several studies have turned toward malware and phishing targeting gaming platforms specifically. Ouellette et al. [22] proposed a detection system for malware targeting online gaming platforms, focusing on bot-driven distribution of malicious content. Sonne and Chindipha [30] proposed an automated system for detecting and preventing phishing attempts targeting Steam accounts, focusing on authentication weaknesses. Kimber et al. [12] characterized scammer behavior and phishing websites targeting Steam accounts, examining the timeline and consequences of account compromise. Smylie et al. [29] proposed a detection approach for identifying Steam bot accounts used to distribute malicious content.

Prior research has established that scams and phishing are prevalent in online gaming ecosystems. Our work builds on this foundation by providing the

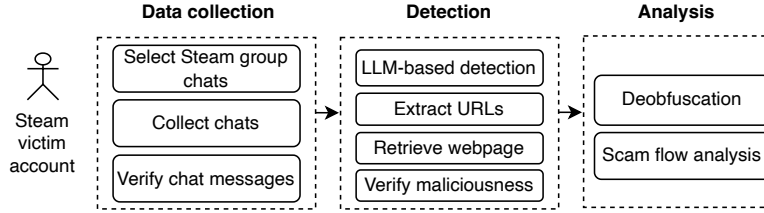


Fig. 1: The overview of the analysis pipeline.

first analysis of the phishing mechanics and evasion infrastructure for attacks targeting Steam environment.

3 Methodology

To understanding evasion mechanics of Steam phishing attacks, we developed an analysis pipeline which includes 1) collection of messages with links that could represent phishing attempts, 2) detection of phishing URLs, 3) network traffic collection and deobfuscation.

This passive observation strategy enabled us to capture unfiltered scam activity over an extended period, providing insights into the dynamics of these attacks without directly interacting with the scammers. The flow of the analysis pipeline is given in Figure 1.

3.1 Data collection

To collect scam messages, we subscribed to 1,250 Steam groups: 706 official game groups and 544 general public groups. We selected all top-ranked groups based on community leaderboard rankings [31], prioritizing official game groups. General public groups were identified using keywords such as "tournament" and "trading" to capture communities centered around item exchange. A dedicated Steam account was created solely for data collection purposes. Using this account, we collected messages between 7 July 2025 and 5 May 2026 (125 days as script terminated due to power outages). To automate collection, we implemented a local Steam client emulator using the `node-steam-user` package [4], combining Node.js for real-time message monitoring with Python for data processing and analysis. Full emulation of the official Steam client ensured real-time, complete capture of every group-chat message. To validate collection, we manually cross-checked a sample of groups against the official Steam client, confirming that our bot received an identical message stream. The collection pipeline was designed to minimize the window between message posting and link analysis to detect phishing infrastructure before potential takedowns or URL expiration. The system included three coordinated components: a primary Node.js bot that maintained a persistent Steam session, joined all subscribed groups, and listened

Below is a list of links, they may or may not point to phishing websites.
 Analyze the links and output the results in the following format:
 field 1-> link provided by me
 field 2-> your assessment of the link (benign, phishing, other types of malicious activity, unknown)
 field 3-> is this a valid HTTP or HTTPS URL?
 field 4-> date when you made this decision (date in ISO format)
 field 5-> rate your confidence level in your link assessment (integer between 0 and 10)

Fig. 2: LLM prompt to identify phishing links.

for incoming messages; secondary Python scripts invoked via `python-shell` to store messages and metadata; and hourly Python scripts for link extraction, deduplication, validation, and resolution. Link inspection was handled using Selenium-Wire, enabling deep inspection of redirects and obfuscated URLs. Key dependencies included `steam-user` (v5.2.3) for Steam client emulation, `steamid` (v2.1.0) for SteamID parsing, and `python-shell` (v5.0.0) for cross-language integration. All components ran locally on a single machine.

3.2 Detection

A substantial proportion of communication within Steam chat groups is benign and does not involve malicious content. Since our study examines the live phishing attack campaigns, all collected messages were first preprocessed, and only those containing URLs were forwarded to the detection module. The initial detection of malicious messages and URLs was performed using queries to the Grok, Gemini and Deepseek AI models. Prior research showed the effectiveness of LLMs in the automated detection of phishing URLs [15, 3, 32]. Following this approach, we leverage an LLM-based analysis to expedite the detection process.

Our preliminary analysis revealed that Steam scam messages are frequently duplicated and disseminated at a high rate which could go as high as more than 1000 times in a single hour, indicating automated or coordinated campaigns. To reduce redundancy, we prioritized unique source URLs posted on Steam groups on an hourly basis, as repeated messages often pointed to the same underlying malicious infrastructure.

After parsing the collected messages for links, each link was passed to the LLM models with a prompt instructing the models to detect the links that appear malicious. An example of our prompt is shown in Figure 2. The links detected as malicious by at least one LLM with confidence level above 7 were then manually verified using VirusTotal. Although the confidence value is not a ground-truth probability, it is a useful heuristic ranking signal, sufficient to separate cases when LLM is fairly sure from LLM is guessing, making it more effective for pre-filtering malicious candidates.

For each link identified as malicious, our data collection module (1) verified that the domain was still active, and then (2) downloaded website content. Given the inherently transient nature of malicious infrastructure, the verification was performed as soon as possible after detection to maximize the likelihood of capturing live content.

Table 1: Collected data statistics

607 832	Total messages
377 966	User messages
159 861	Unique user messages
31 043	Unique source links extracted from user messages
107	Unique verified phishing source links
21 127	Corresponding resolved links
570	Unique phishing resolved links

The collection module was implemented in Python and leveraged Selenium, an open-source framework for browser automation, to retrieve dynamically generated web content. By using a headless browser configuration, the system was able to fully render webpages, including client-side elements that are often critical in phishing attacks. Selenium’s configurable settings also enabled requests to be routed either directly or through proxy servers when necessary. After rendering each page using the Selenium WebDriver, all embedded URLs were extracted, and associated resources were downloaded using the requests-futures library. The content of all the collected webpages was hashed using MD5 before storing to keep track of unique webpages.

3.3 Analysis

Once we verified the link and the corresponding website as phishing, the next step was to observe the flow of scam and collect the network traffic. To achieve this, we utilized OWASP ZAP, an open-source web application security testing tool, to intercept and record all communications between the client and the phishing server.

Using OWASP ZAP, we visited phishing URLs through a browser configured to use ZAP as a proxy. We then performed manual interactions with the page, e.g., entering Steam account credentials, authentication code, necessary to trigger the execution of key JavaScript functions and generate relevant network traffic.

4 Phishing links analysis

During our monitoring period, 328 groups received user messages containing scam links, of which 262 were official game groups. In total, we collected 607,832 messages over 125 days. Of these, 377,966 (62%) were user messages, the remainder were server-generated administrative messages not relevant to our analysis. Of the 377,966 user messages, 159,861 (42%) were unique.

From these user messages, we extracted 190,777 source links, of which 31,043 (16%) were unique. Of the 31,043 unique source links, only 248 (0.8%) were flagged as scam by at least one LLM model with a confidence level above 7, and 91 were flagged by all three models. Subsequent verification via VirusTotal

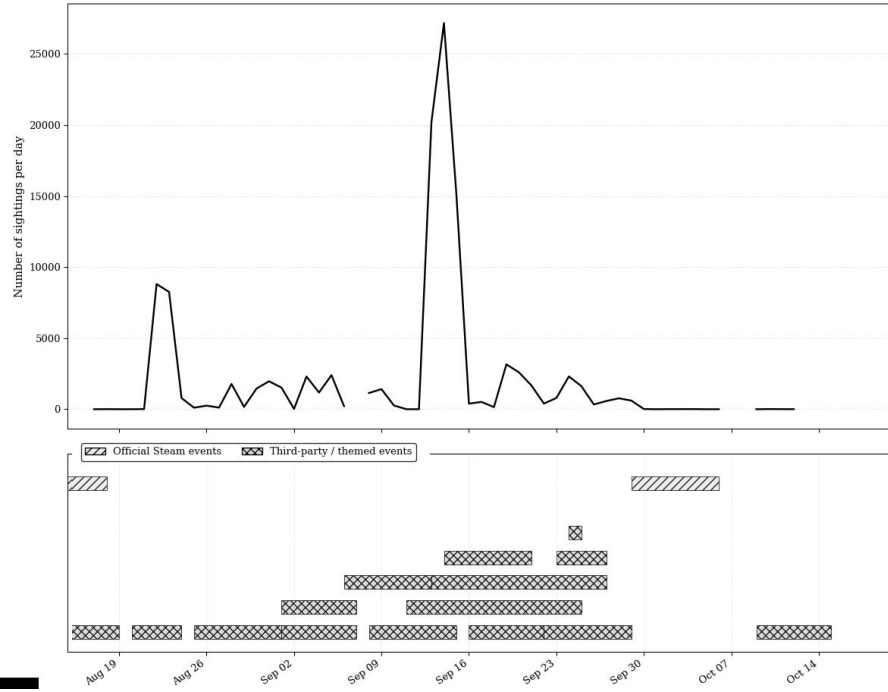


Fig. 3: Appearance of source scam links vs Steam events (July–September 2025)

identified 107 true positives and 141 false positives. Overall, of the 190,777 total source links, 127,155 led to phishing content. Despite there being only 107 unique malicious source links, these accounted for 67% of all source links collected, indicating a high degree of link reuse across messages.

Finally, the 107 unique malicious source links resolved to 570 unique destination URLs. Of these, 504 were accessible via Selenium. Notably, some of the source links were observed resolving to different destination URLs at different points in time.

4.1 Maintaining persistence

Link persistence. Of the 107 confirmed phishing source links, 48 employed typosquatted Steam domain names and the rest used URL shorteners. Typosquatting is a common technique in which attackers register domain names that closely mimic legitimate ones, differing by a single character, transposition, or substitution, to deceive users who mistype or fail to explore a URL carefully.

These links showed varying degrees of persistence across the 125-day collection period. Most appeared on only one or a few days, however, a small subset demonstrated sustained activity over multiple weeks. The most persistent link, <https://goto.now/SteamsGifts>, was observed on 13 distinct days, followed

by `https://u.to/3sJLIg` (8 days) and several `goto.now` variants active on 6–8 days each.

Temporal event correlation. Daily phishing link sightings exhibited clear temporal clustering, with sharp peaks coinciding with both official Steam promotions and third-party gaming events (Figure 3). The most dramatic spike occurred on September 14, with 27,154 sightings of `https://goto.now/SteamGifts`, aligning with multiple events including 'The Hungry Ghost Festival' (6th September - 13th September) and 'Wonders of Wizardry Sale 2025' (September 11th – September 25th), 'Mexican Entertainment System' (14th September - 21st September). A secondary cluster of increased activity was observed during August 22–25, coinciding with the FGS Gamescom event and the Back to School Games Celebration. Several links, particularly those hosted on `u.to` and `goto.now`, sustained high daily volumes across multiple days, with single-day counts exceeding 5,000 in several instances (e.g., 5,267 appearances of `https://u.to/3sJLIg` in a single day). This pattern strongly suggests that scammers deliberately time campaigns to exploit periods of heightened user engagement during sales and themed events, when players are more likely to click links promising free or discounted items.

Evasion infrastructure. A key characteristic of the observed scam infrastructure was the diversity of resolved destinations behind a limited set of source links. Of the 570 unique resolved destination URLs, 492 originated from just nine source links that each appeared more than 1,000 times. Furthermore, 129 unique resolved links were active across multiple days, with some reappearing as late as 20 days after their first observation.

The most heavily reused source link, `https://goto.now/SteamsGifts`, resolved to 354 distinct destination URLs; `https://goto.now/SteamGifts` resolved to 167, and `https://steamcommunllity.com/activatlon=Uot4Fbgs2a` to 159. While most source links resolved to only one or two destinations, the top 10 sources accounted for the vast majority of destination diversity. This concentration indicates that scammers rely on a small, stable set of entry-point URLs, while continuously rotating back-end destinations to evade detection and extend campaign lifespan. The pattern is consistent with the use of modular phishing kits capable of rapidly generating new redirect chains while reusing established front-end entry points.

5 Phishing mechanics analysis

We collected and analyzed the web content of 504 unique resolved phishing URLs, retrieving 45,077 files in total across a broad range of types, including HTML, binary, GZip, PNG, JPEG, PEM, SVG, and font formats (TTF, WOFF), among others (Table 2). Despite this volume, only 8,646 files (19%) were unique by content hash, indicating substantial reuse of assets across phishing sites. In terms of composition, binary and GZip files together accounted for



Fig. 4: The phishing attack flow.

Table 2: Distribution of collected files served by phishing attacks

File Type	Total	%	Unique	%
BINARY	16,381	37.7%	1,549	17.9%
GZIP	7,576	16.8%	952	11.0%
TEXT	6,452	14.3%	2,021	23.4%
HTML	5,747	12.7%	3,026	35.0%
PNG	3,393	7.5%	327	3.8%
PEM	1,232	2.7%	8	0.09%
ZST	767	1.7%	387	4.5%
LZ4	735	1.6%	102	1.2%
ICO	638	1.4%	39	0.5%
JPEG	517	1.1%	117	1.4%
SVG	352	0.8%	27	0.3%
TTF	315	0.7%	6	0.07%
GIF	217	0.5%	39	0.5%
RIFF	133	0.3%	38	0.4%
WOFF	4	0.009%	3	0.03%
EMPTY	13	0.03%	1	0.01%
AUDIO	2	0.004%	1	0.01%
PGP	2	0.004%	2	0.02%
SGML	1	0.002%	1	0.01%
Total	45,077	100%	8,646	100%

over 50% of all collected files, followed by HTML at 13% and PEM certificate files at 3%. The high proportion of binary and compressed content is consistent with deliberate obfuscation of phishing kit assets.

Examination of the HTML files revealed obfuscated Javascript code, with five distinct obfuscation variants identified across the dataset. Interestingly, these variants were not deployed simultaneously. These variants were rotated, i.e., each was active for a period before being replaced by another. Some scripts reappeared multiple times over our collection period. Despite their structural differences, all five variants implemented the same functionality: token-based phishing kits designed to harvest Steam session credentials.

5.1 Deobfuscating JavaScript

The obfuscated JavaScript files employed multiple layers of obfuscation that defeated all standard automated deobfuscation tools: JStillery [16], js-deobfuscator [27], de4js [14], illuminateJS [23], and deobfuscate.relative.im [25], primarily due to their reliance on runtime-generated values and encrypted string arrays that require execution context to resolve. One script was additionally delivered with syntax errors, requiring manual correction before analysis could proceed. As a result, all scripts required manual reverse engineering. Across the scripts, we identified the following obfuscation techniques:

- **Identifier renaming.** Variable and function names were replaced with short, meaningless identifiers.
- **String array indirection with caching and rotation.** Meaningful strings were replaced with numeric indices into a large obfuscated array, retrieved via a lookup function. In some scripts, the array had continuous rotation via a self-executing loop, making static index-to-string mapping impossible without executing the rotation logic.

- **Custom Base91 encoding.** Strings stored in the obfuscated array were encoded using a custom Base91-style algorithm with a unique 91-character alphabet per decoder, mapping characters to 13- or 14-bit values before unpacking into 8-bit bytes.
- **Control flow obfuscation.** Sequential control flow was flattened using infinite loop dispatchers with opaque arithmetic checksums, and deeply nested ternary operators with multiple runtime-dependent exit points.
- **Dead code injection via library embedding.** Complete legitimate libraries including `Interact.js`, `il8next`, a QR code generator, and React JSX runtime helpers, were interleaved with malicious logic, creating a large amount of noise that overwhelmed automated analysis tools.
- **Opaque predicates and arithmetic encoding.** Numeric literals were encoded as hexadecimal values or embedded in complex arithmetic expressions. Conditional checks combined type coercion, property existence checks, and string manipulation whose values were only resolvable at runtime.
- **Dynamic string reconstruction via XOR cipher.** API endpoint strings (e.g., `/steam/doLogin`, `/steam/submitCode`) were absent from the source and reconstructed at runtime using an XOR cipher keyed on `window.localStorage.secret`, rendering static analysis insufficient without knowledge of the runtime value.
- **Self-modifying code.** The primary string lookup function redefined itself after its first invocation.
- **Polymorphic decoders.** Multiple independent decoding functions each used a distinct Base91 alphabet, bound to specific execution contexts via closure scoping.
- **Function Length Spoofing and Proxy Wrappers.** Several functions forcibly overrode their length property to zero, and key API calls were hidden behind multiple layers of redundant proxy wrappers that add complexity without substantive functionality.

Most scripts used all of the above obfuscation techniques.

Deobfuscation Methodology Given the failure of automated tools, we adopted a four-step dynamic analysis approach:

1. **Unminification.** All scripts were first unminified using an online tool [28], converting single-line minified code into formatted JavaScript with preserved indentation. This exposed control flow structure without decrypting obfuscated strings.
2. **Environment Emulation.** Since the scripts were designed for a specific browser-based environment, we emulated the required context within Node.js by implementing mock objects for `document`, `window`, `fetch`, `XMLHttpRequest`, `localStorage`, `sessionStorage`, and cookie getters and setters, enabling local execution without crashes.
3. **Breakpoint Tracing.** Using VSCode’s debugger, we set breakpoints at critical junctures such as encryption routines, string array accesses, and network

request functions, and stepped through execution to observe how obfuscated constructs resolved to actual values at runtime.

4. **Iterative Replacement.** Each obfuscated function or string was replaced with its deobfuscated equivalent after its runtime output was determined, progressively reconstructing the underlying logic across all three scripts.

5.2 Step-by-step phishing mechanics analysis

After deobfuscating all Javascript files, we were able to reconstruct the phishing attack mechanics that were identical across all HTML and Javascript files we collected (illustrated in Figure 4).

Step 1: Obtaining session key A message containing a phishing link is posted in a Steam group chat. When the victim clicks the link, they are redirected to their browser outside the Steam application. The browser issues a GET request to the resolved phishing URL, hosted on the malicious server, which responds with an HTML page (HTML#1) that renders as a black screen. While appearing blank to the user, this response serves a precise purpose: assigning each visitor a unique session encryption key to track individual compromises. This key is delivered via the HTTP **Set-Cookie** response header in the form of a JWT (JSON Web Token), consisting of Base64-encoded header, payload, and signature components. Obfuscated Javascript embedded in HTML#1 extracts the token payload, which contains a JSON object with three fields: **owner**, **secret**, and **service**. The value of the **secret** field is stored in the browser’s global **window.token** object, making it globally accessible throughout the session. We hereafter refer to this value as the *encryption key*.

Obfuscated Javascript#1 then constructs a hidden iframe containing an invisible HTML form. The script collects three pieces of browser context: the URL pathname (**window.location.pathname**), the query string (**window.location.search**), and the hostname of the phishing domain (**window.location.hostname**). Each value is inserted into a separate hidden input field within the form. The script then retrieves the encryption key from **window.token.secret** and applies XOR encryption to **window.location.pathname**, producing what we refer to as encrypted *path#2*, to aid clarity in tracing the subsequent steps of the attack flow.

Step 2: Forged interface delivery The hidden form is submitted via an HTTP POST request to *path#2*, transmitting the pathname, query string, and hostname of the resolved phishing URL to the malicious server. The form is then removed from the local DOM after 800 ms.

The server responds with HTML#2, which marks the first point of visible user interaction: a forged Steam gift card page designed to visually impersonate Valve Corporation’s official promotional layout, complete with UI elements such as a “Sign In” button (Figure 5). Unlike HTML#1, this file includes



Fig. 5: The Giftcard Page

hardcoded references to two external Javascript files, which are requested asynchronously from the phishing server. Javascript#2 loads successfully, however, requests for Javascript#3 consistently returned a `Not Found` in JSON format response across all tested links. Javascript#2 largely replicates the key-extraction flow of Javascript#1, re-reading the encryption key from `window.token.secret`. It additionally stores a key in `window.localStorage.secret`. Note that both steps redundant given the key is already available locally. The script then performs three encrypted operations in sequence:

1. XOR-encrypts the string `"/getSiteConfig"` with the encryption key, producing `path#3`, and issues an HTTP GET request to that path. The server responds with a JSON object containing site configuration settings (Figure 6), which are subsequently used to construct the fake login iframe.
2. XOR-encrypts the string `"/pageVisit"` with the encryption key, producing `path#4`, and issues an HTTP POST request to that path. The server responds with `"success": true`.

All of the above occurs without any further user interaction beyond the initial click on the phishing link posted in the Steam group chat.

Step 3: Login form When the victim clicks “Sign In” or any other UI element on the forged gift card page, Javascript#2 is triggered once more. It XOR-encrypts the string `"/FakeLoginPage"` using the encryption key retrieved from `window.token.secret`, producing `path#5`, which is stored in the global object `window.loginURL`. All interactive elements on the phishing page are bound to a single event listener that fires a `message` event on the current window upon any button click. Javascript#2 responds to this event by reading the `type` field from the previously retrieved site configuration settings and invoking the corresponding display function (Figure 7), which determines how the fake Steam login window is rendered and presented to the victim.



Fig. 6: Configuration settings delivered in Step 2.

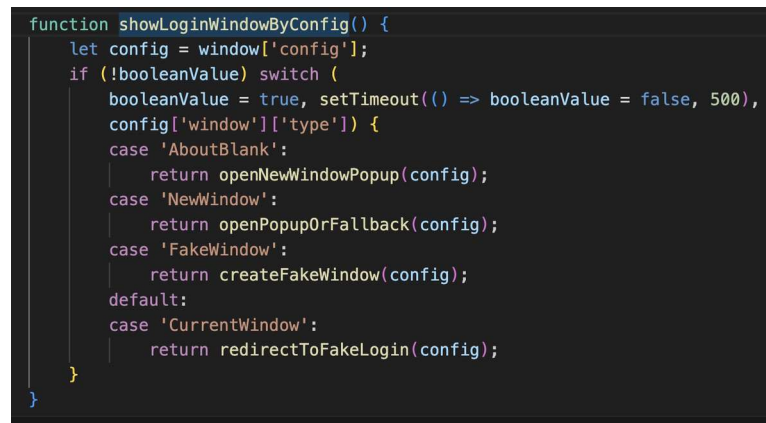


Fig. 7: Javascript#2 function for rendering the fake login window based on server configuration settings

The display function passes the full configuration settings along with `window.loginURL` (containing *path#5*) into a template generator, which embeds the path as the `src` attribute of an HTML iframe string.

The code objects the iframe into the DOM, and consequently an HTTP GET request to *Path#5* is issued. The server responds with HTML file#3, which contains a hardcoded reference to obfuscated Javascript file#4. A subsequent GET request retrieves Javascript#4 from the malicious server, which renders a fake Steam login form inside the iframe, presented to the victim as a convincing popup window visually identical to the legitimate Steam login page (Figure 8).

Concurrently with the login form rendering, Javascript file #4 XOR-encrypts the string `"/steam/visit"` using the encryption key retrieved from `window.localStorage.secret`. Notably, this is the first Javascript file in the chain that reads the encryption key from `localStorage` without parsing it from the `Set-Cookie` header first, as all preceding scripts had done. The resulting *path#6* serves as a “visit” endpoint, i.e., Javascript#4 issues an HTTP POST

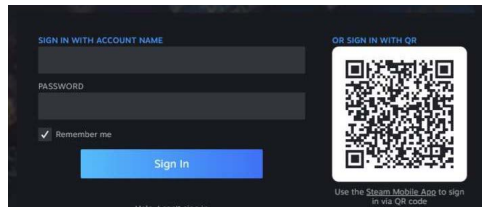


Fig. 8: Fake Steam login page

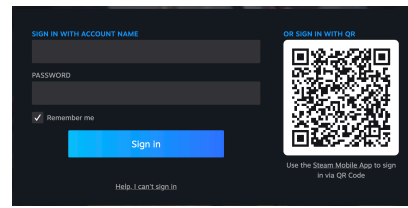


Fig. 9: Legitimate Steam login page

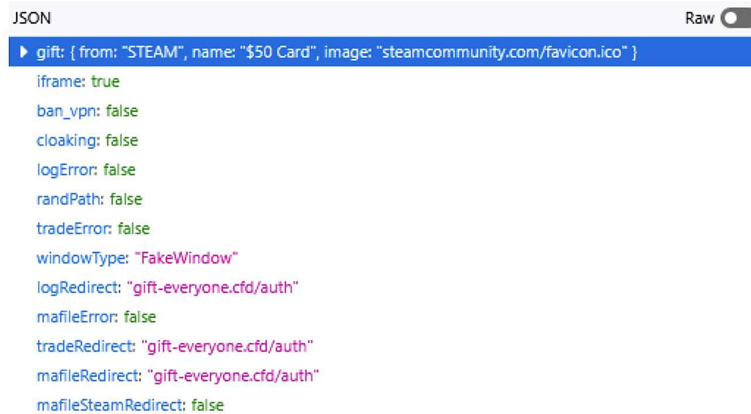


Fig. 10: Server-returned gift card configuration parameters

request to this path, notifying the attacker’s server that a victim has reached the login form. The script also checks `localStorage` for a `visit` flag to distinguish first-time visitors from returning ones. In response, the server provides configuration details about the gift card offering (Figure 10). At this point, no further activity occurs unless the victim chooses to enter their credentials and click “Sign In.”

Step 4: Credential Harvesting When the victim submits their username and password, Javascript#4 XOR-encrypts the string `"/steam/doLogin"` using the key from `window.localStorage.secret`, producing `path#7`. The browser issues an HTTP POST request to this path containing the submitted credentials, and the server responds with a JSON object indicating whether the login attempt was successful. To characterize the credential harvesting behavior, we conducted five controlled login sessions using three real Steam accounts, each containing funds. Two accounts had email-based two-factor authentication enabled, the third had no two-factor authentication configured. One additional session used fake credentials as a baseline.

Session 1: Fake credentials. Submitting an invalid username and password produced a JSON response indicating authentication failure, reflected on screen as an invalid password error.

Session 2: Valid credentials, email authenticator, fake code. Submitting valid credentials (email authenticator enabled) prompted a two-factor authentication screen. Entering a fake code triggered a POST request to encrypted path (XOR of `"/steam/submitCode"`), returning a JSON response indicating an invalid code. The account was not compromised.

Session 3 and 4: Valid credentials, email authenticator, correct code. Submitting the correct authentication code for valid credentials returned the same “invalid code” response. Despite this, the account’s email authenticator was silently removed, and the account was taken over approximately one hour later. We have conducted this step for two different valid accounts with email authenticator enabled.

Session 5: Valid credentials, no email authenticator enabled. Despite an account having no email authenticator, the phishing page still presented a two-factor authentication prompt. Submitting a fake code returned the same invalid code response, and the account was taken over immediately.

6 Discussion

Our findings reveal several key observations about the structure and sophistication of phishing operations within the Steam ecosystem:

- The most striking finding was the extreme concentration of phishing activity among a tiny fraction of users. Just 360 accounts, approximately 1% of users who sent messages, were responsible for all 127,155 source link instances, which constituted roughly a quarter of all user messages. Of these, 19 users alone accounted for 80% of all scam traffic. This degree of concentration strongly suggests automated operations rather than individual human actors, and represents a pattern that platform-level rate limiting or behavioral detection could easily address.
- The reuse of source links further reinforces this interpretation: just 9 source links accounted for 99% of all phishing traffic. Combined with the heavy use of typosquatted Steam domains and URL shorteners, this efficiency mirrors patterns observed in email-based phishing campaigns, adapted to Steam’s group chat environment.
- The phishing infrastructure proved to be quite persistent over time. While most unique scam links appeared briefly, a subset persisted for weeks. `https://goto.now/SteamsGifts` was observed on 13 separate days, and several other links remained active for 6–8 days, suggesting active maintenance of the phishing infrastructure.
- The diversity of resolved destinations behind a small set of persistent source links. For example, 354 distinct destination URLs behind a single shortened entry point reveal a sophisticated evasion strategy. By continuously rotating

back-end destinations while keeping entry-point URLs constant, attackers can circumvent URL-based blocklists while maintaining a single distribution channel, maximizing resilience against takedown efforts.

- The phishing attacks employed sophisticated Javascript obfuscation that defeated standard automated deobfuscation tools, requiring full dynamic analysis with breakpoint tracing and iterative manual replacement. This level of protection not only hinders static analysis and signature-based detection, but also substantially increases the effort required for reverse engineering and takedown, contributing directly to the campaign’s longevity and scalability.
- Analysis of the 504 unique phishing destinations revealed only five structural variants of the phishing kit, all implementing identical credential-harvesting mechanics. This uniformity strongly indicates the use of a shared phishing kit deployed at scale. Notably, one variant was delivered with syntax errors requiring manual correction before it could execute. This suggests either hasty deployment or incomplete kit configuration on the attacker’s part.

7 Conclusion

Phishing within the Steam ecosystem is prevalent. Mitigating these threats is difficult without first understanding the mechanics by which attacks are conducted and the evasion techniques that allow them to persist undetected. This paper presents the first study to systematically analyze the mechanics of phishing campaigns propagating through Steam community groups. By monitoring and analyzing communications across official and third-party Steam groups over 125 days, we reconstructed the full attack chain through traffic interception and manual Javascript deobfuscation.

Our analysis reveals a technically sophisticated and evasive phishing operation. The campaigns employed token-based phishing kits that assigned each victim a unique session encryption key via a JWT delivered in the Set-Cookie header, enabling per-victim tracking and individualized credential harvesting. All sensitive API endpoints were concealed through runtime XOR encryption keyed on session-specific values stored in localStorage, rendering static analysis of network traffic insufficient. The phishing pages rotated continuously across five structurally distinct Javascript obfuscation variants that defeated automated deobfuscation tool we evaluated, requiring full manual dynamic analysis in an emulated browser environment. At the infrastructure level, a small set of persistent entry-point URLs resolved to hundreds of distinct back-end destinations, allowing attackers to evade URL-based blocklists while maintaining stable distribution channels. Campaigns were further timed to coincide with high-traffic Steam promotional events, maximizing victim exposure during periods of elevated user engagement.

Together, these findings demonstrate that phishing operations within the Steam ecosystem have reached a level of technical maturity that renders reactive, user-facing defenses insufficient. Effective mitigation will require proactive behavioral analysis, automated link filtering, and closer cooperation between platform operators and the security research community.

References

1. Bardzell, J., Jakobsson, M., Bardzell, S., Pace, T., Odom, W., Houssian, A.: Virtual worlds and fraud: Approaching cybersecurity in massively multiplayer online games. In: *Proceedings of DiGRA 2007 Conference: Situated Play (2007)*
2. Bitaab, M., Cho, H., Oest, A., Lyu, Z., Wang, W., Abraham, J., Wang, R., Bao, T., Shoshitaishvili, Y., Doupé, A.: Beyond phish: Toward detecting fraudulent e-commerce websites at scale. In: *2023 IEEE Symposium on Security and Privacy (SP)*. pp. 2566–2583 (2023)
3. Chang, Y.C., Aïmeur, E.: “is this site legit?”: Llms for scam website detection. In: *Web Information Systems Engineering – WISE 2024: 25th International Conference, Doha, Qatar, December 2–5, 2024, Proceedings, Part IV*. p. 230–245. Springer-Verlag, Berlin, Heidelberg (2024). https://doi.org/10.1007/978-981-96-0573-6_17, https://doi.org/10.1007/978-981-96-0573-6_17
4. DoctorMckay: Node-steam-user npm package (2024), <https://github.com/DoctorMckay/node-steam-user>
5. Grandhi, S.R., Galimotu, N.C.: Understanding social engineering threats in massively multiplayer online role-playing games: an issue review. *GAP Indian J. Forensics Behav. Sci* **1**(1), 66–71 (2020)
6. Guardio: Steam tops Q1 2025 most imitated brands list. Guardio Blog (2025), <https://guard.io/blog/most-imitated-brands-phishing-scams-q1-2025>, accessed: June 2026
7. Han, M.L., Kwak, B.I., Kim, H.K.: Cheating and detection method in massively multiplayer online role-playing game: systematic literature review. *IEEE Access* **10**, 49050–49063 (2022)
8. Hilven, A., Woodward, A.: How safe is azeroth, or, are mmorpgs a security risk? (2007)
9. Hoang, Q.: Game hacking & anti-cheat analysis. Cybersecurity Undergraduate Research Showcase, Project 3 (2025), <https://digitalcommons.odu.edu/covacci-undergraduateresearch/2025fall/projects/3>
10. Joshi, R.: Cheating and virtual crimes in massively multiplayer online games. Royal University of London (2008)
11. Karkallis, P., Blasco, J., Suarez-Tangil, G., Pastrana, S.: Detecting video-game injectors exchanged in game cheating communities. In: *Computer Security–ESORICS 2021: 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4–8, 2021, Proceedings, Part I* 26. pp. 305–324. Springer (2021)
12. Kimber, J., Branca, E., Stakhanova, N., Gutwin, C.: Understanding Phishing Scams in the Steam Ecosystem. In: *22th EAI International Conference on Security and Privacy in Communication Networks (SecureComm 2026)*. EAI, New York, NY, USA (2026)
13. Lee, K., Lim, K., Kim, H., Kwon, Y., Kim, D.: 7 days later: Analyzing phishing-site lifespan after detected. In: *Proceedings of the ACM on Web Conference 2025*. p. 945–956. WWW ’25, Association for Computing Machinery, New York, NY, USA (2025)
14. lelinhtinh: de4js: Javascript deobfuscator and unpacker. GitHub repository (2021), <https://github.com/weisk/de4js>
15. Mahendru, S., Pandit, T.: Securennet: A comparative study of deberta and large language models for phishing detection. *arXiv preprint arXiv:2406.06663* (2024)
16. (maintainer), S.: Jstillery: Advanced javascript deobfuscation via partial evaluation. GitHub repository (2018), <https://github.com/mindedsecurity/jstillery>

17. Meyer, T.L.: A study on trading scams in massively multiplayer online role-playing games and risk mitigation techniques. Iowa State University (2011)
18. Oest, A., Safaei, Y., Doupé, A., Ahn, G.J., Wardman, B., Tyers, K.: Phishfarm: A scalable framework for measuring the effectiveness of evasion techniques against browser phishing blacklists. In: 2019 IEEE Symposium on Security and Privacy (SP). pp. 1344–1361. IEEE (2019)
19. Oest, A., Safaei, Y., Doupé, A., Ahn, G.J., Wardman, B., Warner, G.: Inside a phisher’s mind: Understanding the anti-phishing ecosystem through phishing kit analysis. In: 2018 APWG Symposium on Electronic Crime Research (eCrime). pp. 1–12. IEEE (2018)
20. Oest, A., Zhang, P., Wardman, B., Nunes, E., Burgis, J., Zand, A., Thomas, K., Doupé, A., Ahn, G.J.: Sunrise to sunset: Analyzing the end-to-end life cycle and effectiveness of phishing attacks at scale. In: 29th USENIX Security Symposium (USENIX Security 20) (2020)
21. Oh, J., Borbora, Z.H., Srivastava, J.: Automatic detection of compromised accounts in mmorpgs. In: 2012 International Conference on Social Informatics. pp. 222–227. IEEE (2012)
22. Ouellette, N., Baseri, Y., Kaur, B.: I am bot the “fish finder”: Detecting malware that targets online gaming platform. In: International Congress on Information and Communication Technology. pp. 261–274. Springer (2023)
23. Paganoni, S., Bronson, S., geeksonsecurity: Illuminatejs: A static javascript deobfuscator. GitHub repository (2018), <https://github.com/geeksonsecurity/illuminatejs>
24. Parizi, R.M., Dehghantanha, A., Choo, K.K.R., Hammoudeh, M., Epiphaniou, G.: Security in Online Games: Current Implementations and Challenges, pp. 367–384. Springer International Publishing, Cham (2019)
25. relative: Synchrony: javascript-obfuscator cleaner & deobfuscator. GitHub repository (2020), <https://github.com/relative/synchrony>
26. Rishiwal, V., Agarwal, U., Yadav, M., Alotaibi, A., Yadav, P., Tanwar, S.: Blockchain-secure gaming environments: A comprehensive survey. IEEE Access **12**, 183466–183488 (2024)
27. ben sb: javascript-deobfuscator: General purpose javascript deobfuscator. GitHub repository (2021), <https://github.com/ben-sb/javascript-deobfuscator>
28. Shape Security: unminify: A tool to unminify javascript, css, html, xml and json code. GitHub repository (2017), <https://github.com/shapesecurity/unminify>
29. Smylie, E., Shiraskar, S., Beyzavi, H., Shiraskar, A., Albluwi, A., Albalawi, M., Nehmetallah, G., Rizk, F., Rizk, P., Rizk, R., Santosh, K., Rizk, D.: High-assurance bot detection on steam: A hybrid ensemble learning framework for detecting automated steam accounts using profile metadata. In: 2026 IEEE Conference on Artificial Intelligence (CAI). pp. 1811–1816 (2026)
30. Sonne, K., Chindipha, S.D.: An automated system for detecting and preventing phishing attempts on steam accounts. In: 2023 International Conference on Electrical, Computer and Energy Technologies (ICECET). pp. 1–6. IEEE (2023)
31. stmlldrr, T.: Steam ladder, group membership (2024), <https://steamladder.com/ladder/groups/>
32. Trad, F., Chehab, A.: Prompt engineering or fine-tuning? a case study on phishing detection with large language models. Machine Learning and Knowledge Extraction **6**(1), 367–384 (2024), <https://www.mdpi.com/2504-4990/6/1/18>
33. Tricomi, P.P., Facciolo, L., Apruzzese, G., Conti, M.: Attribute inference attacks in online multiplayer video games: A case study on dota2. In: Proceedings of the

- Thirteenth ACM Conference on Data and Application Security and Privacy. p. 27–38. CODASPY '23, Association for Computing Machinery, New York, NY, USA (2023)
34. Zhang, P., Oest, A., Cho, H., Sun, Z., Johnson, R., Wardman, B., Sarker, S., Kapravelos, A., Bao, T., Wang, R., Shoshitaishvili, Y., Doupé, A., Ahn, G.J.: Crawlphish: Large-scale analysis of client-side cloaking techniques in phishing. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 1109–1124 (2021)

A Ethical Considerations

This study involves the collection and analysis of active phishing infrastructure targeting Steam users, which raises important ethical considerations that we address throughout our methodology.

Responsible data collection. All data used in this study, including phishing links and associated messages, were collected from publicly accessible sources without any interaction with victims or attempts to engage with threat actors beyond passive observation of credential harvesting behavior. We did not submit any personally identifiable information to phishing infrastructure at any point during our analysis. Where interaction with a phishing kit was necessary to observe its post-click behavior, we used fictional credentials that could not compromise a real account and synthetic credentials for an account created specifically for that purpose and lacking any personal information and funds.

Minimizing risk. Our goal is to document the mechanics and evasion techniques used by Steam phishing infrastructure for the benefit of security researchers, platform operators, and defenders. We therefore report findings at a level of abstraction sufficient to inform detection and mitigation efforts while deliberately withholding implementation code and configuration files, that could allow malicious actors to improve these phishing kits. We believe the value of exposing these post-click phishing mechanics, enabling better detection and platform-level defenses, outweighs the limited and carefully mitigated risks associated with this research.

Responsible disclosure. Our findings identified during this study were reported to Valve’s security team to support detection and mitigation efforts.